2. Entwicklung und Umsetzung von Algorithmen

2.1 Aufgabe 1

Nachfolgend finden Sie ein Lösungsbeispiel als Pseudocode.

Weitere Lösungen in anderen Formen sind möglich.

```
Jahresstatistik statistik(verbrauch: int [][], limit:int)
{
    int minMonat = Integer.MAX_VALUE;
    int maxMonat = Integer.MIN_VALUE;
    List<Monatsverbrauch> greaterLimit = new List<Monatsverbrauch>();
    for (int vn = 0; vn < verbrauch.length; vn++)
        {
        for (int m = 1; m <= 12; m++)
            {
            int v = verbrauch[vn][m+1] – verbrauch[vn][m];
            if (v > limit)
                {
                greaterLimit.add(new Monatsverbrauch( verbrauch [vn][0], m, v));
                }
            if (v < minMonat)
                {
                minMonat = v;
                }
            else if (v > maxMonat)
                {
                maxMonat =v;
                }
            }
        }
return new Jahresstatistik (minMonat, maxMonat, greaterLimit);
}
```

2.2 Aufgabe 2

aa) Unit-Tests können auch als Komponententests bezeichnet werden. Mithilfe dieser Tests wird überprüft, ob alle Einzel-Komponenten (Units) so funktionieren, wie dies beabsichtigt war. Unit-Tests sind voneinander unabhängig und isoliert, ein fehlgeschlagener Unit-Test darf deshalb nicht zu weiteren Fehlern in anderen Units führen. Außerdem sind gute Unit-Tests um eine sogenannte Test Fixture gruppiert und nicht um eine Klasse.

ab) Unit-Tests folgen den F.I.R.S.T.-Prinzipien:

- **F**ast: Die Ausführung der einzelnen Tests sollte schnell sein, damit möglichst oft getestet werden kann.
- **I**ndependent: Da Unit-Tests unabhängig voneinander sind, können sie in beliebiger Reihenfolge und Anzahl parallel oder einzeln ausgeführt werden.
- **R**epeatable: Unit-Tests müssen mehrfach ausführbar sein, während das Ergebnis jedoch immer gleich sein muss.
- **S**elf-Validating: Ein Unit-Test unterstützt nur die Status „fehlgeschlagen“ und „erfolgreich“. Der Status muss automatisch gesetzt werden. Eine manuelle Prüfung darf nicht notwendig sein.
- **T**imely: Unit-Tests sollten vor der Entwicklung des eigentlichen Codes geschrieben werden.

Unit-Tests erfüllen weitere Eigenschaften, daher sind weitere Lösungen denkbar.

©2022 u-form Verlag – Kopieren verboten!

ba)

Test	Ergebnis des Tests
1.	OK
2.	Fehler
3.	Fehler
4.	OK
5.	OK

bb)
```
int berechneDifferenz (int zaehlerstandAlt, int zaehlerstandNeu) {
        int differenz = -1;
        if (zaehlerstandAlt >= 0 && zaehlerstandNeu >= 0) {
                if (zaehlerstandAlt <= zaehlerstandNeu) {
                differenz = zaehlerstandNeu – zaehelerstandAlt;
                }
        }
        return differenz;
}
```

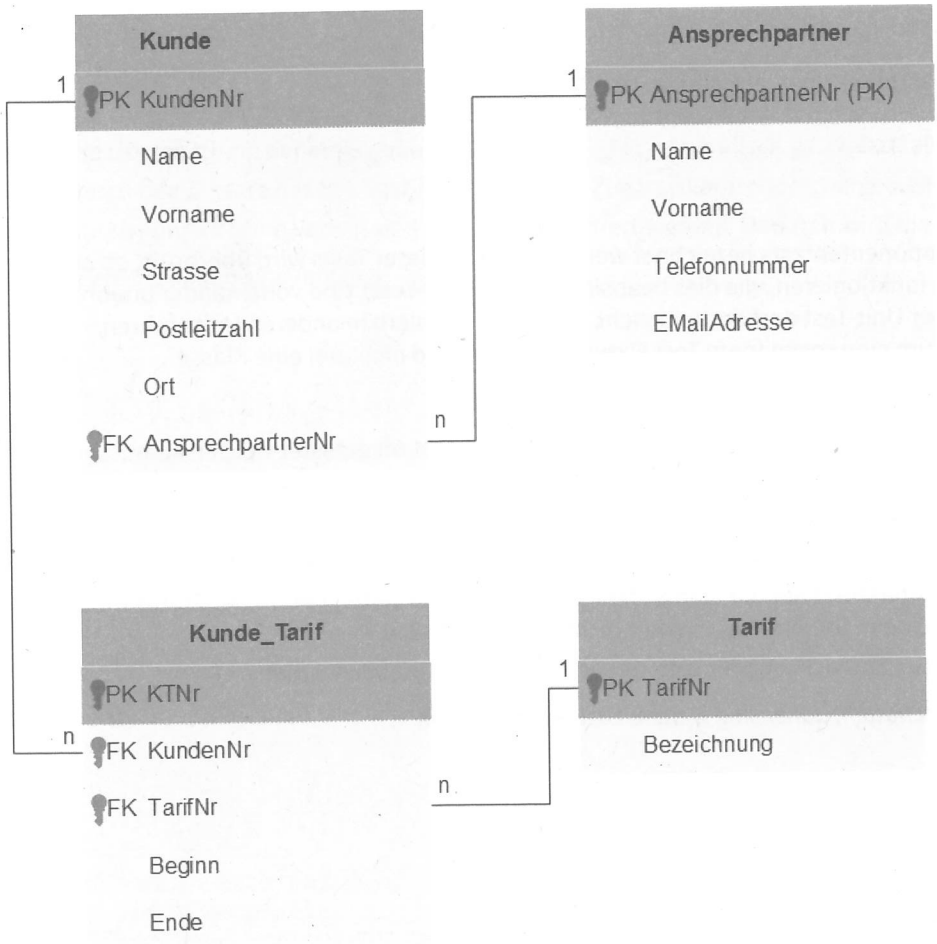
Weitere Lösungen sind möglich.

bc)

Nr.	zaehlerstandAlt	zaehlerstandNeu	differenz
1	-100	80	-1
2	-70	0	-1
3	-22	-5	-1

Siehe Zeile 3 des Codes. Weitere Beispiele sind möglich.

2.3 Aufgabe 3



Weitere Lösungen sind möglich.

©2022 u-form Verlag – Kopieren verboten!

2.4 Aufgabe 4

aa) Wortwörtlich übersetzt, handelt es sich hierbei um eine gespeicherte Prozedur. Hierdurch können komplexe Abfolgen von SQL-Anweisungen unter einem Namen im Datenbank-Management-System (DBMS) abgespeichert werden. Diese Prozeduren können anschließend über den jeweiligen Namen aufgerufen und jederzeit ausgeführt werden.

Eine gespeicherte Prozedur kann von verschiedenen Datenbank-Clients genutzt werden. Änderungen an der Prozedur erfolgen zentral.

ab) Bei einem Trigger handelt es sich um eine spezielle Art einer Stored Procedure. Die Prozedur wird beim Trigger automatisch ausgeführt, sobald ein Ereignis auf dem Datenbankserver auftritt.

ac) Bei einem Datenbankindex handelt es sich um eine Datenstruktur mit Hilfe derer die Abfragegeschwindigkeit gesteigert werden kann. Daher wird die Indizierung genutzt, um insbesondere die Suche nach Datensätzen zu erleichtern und zu beschleunigen. Vor allem bei großen Datenmengen wird hierdurch ein enormer Geschwindigkeitsvorteil ermöglicht.

ba)
```
CREATE INDEX Idx_Datum ON Zaehlerstand (ZSt-Datum);
```

bb)
```
DELETE FROM Haushalte
WHERE ( SELECT Count(Z_ID) FROM Zaehler WHERE Z_HHID = HH_ID ) = 0;
```

Auch möglich:

```
DELETE FROM Haushalte
WHERE HH_ID NOT IN ( SELECT Z_HHID FROM Zaehler);
```

bc)
```
SELECT HH.HH_Nachname AS Nachname, Count(Z.Z_ID) AS AnzZaehler
FROM Haushalte AS HH LEFT JOIN Zaehler AS Z ON HH.HH_ID = Z.Z_HHID
GROUP BY HH.HH_Nachname
ORDER BY Count(Z.Z_ID) DESC;
```

bd)
```
SELECT HH.HH_Nachname AS Nachname, Z.Z_Nummer AS Zaehlernummer,
(( SELECT ZSt.ZSt_Stand FROM ZaehlerStand AS ZSt
WHERE Z.Z_ID = ZSt.ZSt_ZID AND YEAR(ZSt.ZSt_Datum) = 2022) –
(( SELECT ZSt.ZSt_Stand FROM ZaehlerStand AS ZSt
WHERE Z.Z_ID = ZSt.ZSt_ZID AND YEAR(ZSt.ZSt_Datum) = 2021)
) AS Verbrauch
FROM Haushalte AS HH INNER JOIN Zaehler AS Z ON HH.HH_ID = Z.Z_HHID
ORDER BY HH_Nachname;
```

Ebenfalls denkbar:

```
SELECT HH_Nachname AS Nachname, Z_Nummer AS Zaehlernummer,
        (Stand2022- Stand2021) AS Verbrauch

FROM Haushalte INNER JOIN Zaehler ON HH_ID = Z_HHID
        INNER JOIN
        (       SELECT ZSt_ZID, ZSt_Stand AS Stand2022
                FROM ZaehlerStand
                WHERE YEAR(ZSt_Datum) = 2022
        )AS qry22 ON Z_ID =qry22.ZSt_ZID
        INNER JOIN
        (       SELECT ZSt_ZID, ZSt_Stand AS Stand2021
                FROM ZaehlerStand
                WHERE YEAR(ZSt_Datum) = 2021
        )AS qry21 ON Z_ID =qry21.ZSt_ZID
ORDER BY HH_Nachname;
```

©2022 u-form Verlag – Kopieren verboten!